

MARCO TEÓRICO

Transformaciones en Estructuras de Datos Arbóreas y Árboles de Decisión

1. LAS ESTRUCTURAS ARBÓREAS COMO BASE

1.1 ¿Qué es un árbol en programación? Un árbol es una forma de organizar datos de manera jerárquica, donde cada elemento (nodo) puede conectarse con otros elementos que están por debajo de él (hijos). A diferencia de una lista lineal, un árbol permite representar relaciones de dependencia, clasificación o decisiones que se ramifican.

Los grupos anteriores han establecido los conceptos fundamentales:

- Un árbol tiene un **nodo raíz** de donde parte todo.
- Los nodos sin hijos se llaman **hojas**.
- Los nodos con hijos se llaman **internos**.
- Las conexiones entre nodos son las **ramas** o **aristas**.
- Existen tipos especializados como el **árbol binario** (máximo dos hijos por nodo) y los **recorridos** (preorden, inorden, postorden) que permiten visitar todos los nodos en un orden determinado.

1.2 ¿Por qué transformar un árbol? Un árbol no es una estructura estática. En la práctica, los árboles se construyen, se modifican y se adaptan según las necesidades del problema que resuelven. Las transformaciones existen porque:

- Un árbol puede volverse ineficiente si crece desbalanceado.
- Un árbol puede contener partes innecesarias que conviene eliminar.
- Un árbol puede necesitar ser representado de otra forma para transmitirse, visualizarse o interpretarse.

2. TRANSFORMACIONES EN ESTRUCTURAS ARBÓREAS

Una **transformación** es cualquier operación que recibe un árbol como entrada y produce un árbol modificado como salida. La clave está en que la operación **conserva el sentido de los datos**, aunque cambie la forma.

Clasificamos las transformaciones en tres grandes grupos según su propósito.

2.1 Transformaciones estructurales: cambiar la forma del árbol Estas transformaciones reorganizan los nodos, modificando quién es padre de quién, pero manteniendo las relaciones lógicas entre los datos.

a) Rotaciones Concepto: Imagina un árbol que ha crecido demasiado hacia un lado. Una rotación es un “giro” local que cambia la raíz de un subárbol sin alterar el orden de los elementos.

Lógica de programación: - Se reasignan punteros entre un nodo, su hijo y el subárbol que se mueve. - Operación local: solo afecta a tres nodos y sus subárboles inmediatos. - Tiempo constante, ya que es un número fijo de pasos sin importar el tamaño del árbol.

¿Cuándo se usa? En árboles autobalanceables (AVL, Red-Black), donde cada inserción o eliminación puede requerir una rotación para mantener la eficiencia de búsqueda.

b) Conversión de árbol general a árbol binario Concepto: Un árbol donde un nodo puede tener muchos hijos no encaja directamente en una estructura binaria. La conversión permite representar cualquier árbol usando solo dos apuntadores por nodo.

Lógica de programación: Se aplica la técnica “hijo izquierdo, hermano derecho”: - El enlace izquierdo apunta al **primer hijo** del nodo. - El enlace derecho apunta al **siguiente hermano** del mismo nivel.

De esta forma, una lista de hermanos se recorre horizontalmente por los enlaces derechos, y cada uno puede descender verticalmente por su enlace izquierdo.

c) Reorganización por cambio de raíz Concepto: En ciertos contextos puede interesar cambiar cuál es el nodo raíz del árbol, reordenando toda la jerarquía para que un nodo que antes era interno pase a ser la nueva raíz.

Lógica de programación: Implica invertir las relaciones padre-hijo a lo largo del camino entre la raíz original y la nueva raíz, reasignando los subárboles laterales para que no se pierda ninguna rama.

2.2 Transformaciones reductivas: simplificar el árbol Estas transformaciones buscan un árbol más pequeño, eliminando lo que sobra o fusionando lo que se repite.

a) Poda o pruning Concepto: Consiste en cortar una rama completa del árbol, convirtiendo el nodo donde se corta en una hoja. Es como decir: “todo lo que estaba debajo de aquí ya no me interesa, me quedo con lo que tengo hasta este punto”.

Lógica de programación: 1. Se selecciona un nodo interno. 2. Se elimina todo su subárbol (liberando memoria si es necesario). 3. Se marca el nodo como hoja, asignándole un valor final representativo de lo que había antes (por ejemplo, el valor más frecuente entre las hojas eliminadas).

¿Cuándo se usa? Cuando una parte del árbol es demasiado específica, tiene pocos datos asociados o no mejora el resultado general. Es fundamental en árboles de decisión para evitar el sobreajuste.

b) Fusión de nodos equivalentes Concepto: Si dos o más ramas del árbol conducen exactamente al mismo resultado, no tiene sentido mantenerlas separadas. Se fusionan en una sola.

Lógica de programación: 1. Se comparan los valores de las hojas o las estructuras de los subárboles. 2. Si son idénticos, se redirigen todos los punteros entrantes hacia una única copia del nodo o subárbol. 3. La estructura deja de ser un árbol puro y se convierte en un **grafo acíclico dirigido** (DAG), donde un mismo nodo puede tener varios padres.

Beneficio: Ahorro de memoria y simplificación visual y lógica del modelo.

2.3 Transformaciones de representación: cambiar cómo se ve o se consume el árbol Aquí no se altera la forma lógica del árbol, sino el formato en que se entrega o se interpreta.

a) Recorridos como transformación a secuencia Concepto: Un árbol es una estructura bidimensional (ramificada). Aplicar un recorrido significa “aplanarlo” en una secuencia unidimensional de nodos, siguiendo un orden específico.

Lógica de programación: - **Preorden:** Visitar la raíz, luego el subárbol izquierdo, luego el derecho. Útil para copiar o reconstruir el árbol. - **Inorden:** Visitar izquierdo, raíz, derecho. En un ABB, produce los elementos en orden ascendente. - **Postorden:** Visitar izquierdo, derecho, raíz. Útil para eliminar el árbol (borrar hijos antes que al padre) o evaluar expresiones.

Cada recorrido puede implementarse recursivamente o con una pila explícita.

b) Serialización Concepto: Convertir el árbol completo en una cadena de texto o secuencia de bytes para guardarlo en disco o enviarlo por red, con posibilidad de reconstruirlo después.

Lógica de programación: 1. Se elige un recorrido (por ejemplo, preorden). 2. Se escribe cada nodo visitado, usando un marcador especial para indicar “aquí no hay nodo” (nulo). 3. Para reconstruir, se lee la secuencia y se crean nodos en el mismo orden, usando los marcadores para saber cuándo detener una rama.

c) Conversión a reglas Concepto: Cada camino desde la raíz hasta una hoja puede expresarse como una regla condicional (estructura IF-THEN). Esta transformación es particularmente importante en árboles de decisión.

Lógica de programación: 1. Se recorre el árbol desde la raíz. 2. Para cada hoja, se acumulan las condiciones encontradas en los nodos intermedios. 3. Se genera una regla con todas las condiciones unidas por AND y el valor de la hoja como conclusión.

El conjunto de reglas resultante es equivalente al árbol original en cuanto a las decisiones que toma.

3. EL ÁRBOL DE DECISIÓN COMO ESTRUCTURA APLICADA

3.1 Concepto y propósito Un **árbol de decisión** es un tipo especial de árbol cuya finalidad no es almacenar datos para buscarlos, sino **modelar un proceso de toma de decisiones**. Responde a la pregunta: “dadas ciertas condiciones, ¿qué decisión debo tomar?”.

Cada nodo interno representa una **pregunta** sobre una característica observable del problema. Las ramas son las **posibles respuestas**. Las hojas son las **decisiones finales**.

3.2 Estructura lógica A diferencia de un árbol binario de búsqueda (donde los valores determinan la posición), en un árbol de decisión:

- Los nodos contienen **condiciones o pruebas**.
- Las ramas están etiquetadas con los posibles **resultados** de esas pruebas.
- Las hojas contienen la **clase, categoría o acción** a tomar.
- No existe una propiedad de orden entre padre e hijos como en los ABB; la relación es puramente semántica.

3.3 Construcción del árbol de decisión La construcción es en sí misma una **transformación algorítmica de datos a árbol**. El proceso lógico es:

1. Se parte de un conjunto de ejemplos con atributos y una clase conocida.
2. Se selecciona el atributo que mejor separa los ejemplos en grupos (el más informativo).
3. Se crea un nodo con ese atributo y una rama por cada valor posible.
4. Para cada rama, se repite el proceso con los ejemplos que caen en ella.
5. El proceso se detiene cuando todos los ejemplos en una rama pertenecen a la misma clase o no quedan atributos por preguntar.

Este mecanismo es un **algoritmo recursivo de particionamiento** que divide el espacio de datos en regiones cada vez más puras.

3.4 Comparación con otros tipos de árboles

Característica	Árbol binario de búsqueda	Árbol de decisión
Propósito	Búsqueda eficiente	Clasificación o decisión
Contenido del nodo	Valor de dato	Pregunta sobre un atributo
Ramas	Máximo dos (izq/der)	Tantos como valores del atributo
Hojas	Vacías o sin significado especial	Contienen la decisión final
Orden	Estricto (menor a izq, mayor a der)	No hay orden, hay secuencia lógica

4. LAS TRANSFORMACIONES EN EL CONTEXTO DEL ÁRBOL DE DECISIÓN

El árbol de decisión es el escenario ideal para aplicar las transformaciones vistas, porque su utilidad práctica depende directamente de ellas.

4.1 Poda: la transformación más importante en árboles de decisión

Problema que resuelve: Un árbol de decisión construido hasta el detalle máximo tiende a memorizar los ejemplos en lugar de aprender patrones generales (sobreajuste). Esto hace que falle con datos nuevos.

Lógica de la transformación: 1. Se recorre el árbol construido. 2. Para cada nodo interno, se evalúa si su subárbol realmente aporta capacidad de decisión o solo añade complejidad. 3. Si el subárbol es innecesario, se corta y el nodo se convierte en hoja con la clase más probable en ese punto.

Efecto en el programa: El árbol resultante es más pequeño, más rápido de ejecutar y comete menos errores con datos desconocidos.

4.2 Conversión a reglas: transformación para la interpretabilidad

Problema que resuelve: Un árbol complejo puede ser difícil de leer o integrar en un sistema que no maneje estructuras arbóreas.

Lógica de la transformación: 1. Se extrae cada camino completo desde la raíz hasta una hoja. 2. Se construye una regla condicional por cada hoja. 3. Las reglas pueden simplificarse eliminando condiciones redundantes.

Resultado: Un conjunto plano de reglas lógicas equivalente al árbol, que puede ejecutarse secuencialmente o integrarse en bases de conocimiento.

4.3 Reordenamiento de atributos: transformación estructural por criterio de costo

Problema que resuelve: No siempre el primer atributo más

informativo es el más barato o rápido de medir. En un escenario real, puede convenir preguntar primero cosas fáciles de verificar.

Lógica de la transformación: 1. Se analiza el árbol actual y se identifica otro posible orden de preguntas que respete la lógica de las decisiones. 2. Se reestructuran los nodos para que el atributo deseado quede en la raíz o en niveles superiores. 3. Se ajustan las ramas para mantener la coherencia de las rutas de decisión.

5. SÍNTESIS DEL MARCO TEÓRICO

La relación entre los conceptos expuestos se puede resumir en el siguiente flujo lógico:

ÁRBOL COMO ESTRUCTURA DE DATOS

Propiedades básicas: jerarquía, recursividad, nodos y ramas
Tipos: general, binario, de búsqueda, balanceado
Recorridos: preorden, inorden, postorden

TRANSFORMACIONES SOBRE ÁRBOLES

Estructurales: cambiar la forma
Rotaciones (balancear)
Conversión general → binario (unificar representación)

Reductivas: eliminar lo innecesario
Poda (simplificar, evitar sobreajuste)
Fusión de equivalentes (compactar, compartir)

De representación: cambiar el formato
Aplanamiento por recorridos
Serialización (guardar/transmitir)
Conversión a reglas (interpretar, integrar)

CASO DE APLICACIÓN: ÁRBOL DE DECISIÓN

Estructura: nodos-pregunta, ramas-respuesta, hojas-decisión
Construcción: particionamiento recursivo de datos
Transformaciones clave que le dan valor práctico:
Poda → evita sobreajuste
Reglas → facilita comprensión e integración
Reordenamiento → adapta a costos reales

Diferencia fundamental: no busca datos, modela decisiones

Division de Integrantes

PERSONA 1: COORDINADOR/A Y APERTURA DE LA CLASE

Rol: Líder de la exposición y encargado del inicio.

Responsabilidades en la exposición: - Dar la bienvenida y presentar al grupo. - Contextualizar el tema conectando con lo que expusieron los grupos anteriores (árboles, árboles binarios, recorridos pre/in/post). - Explicar **qué son las transformaciones en estructuras arbóreas** y por qué son necesarias. - Presentar la clasificación general de transformaciones (estructurales, reductivas, de representación).

Responsabilidades en el Kahoot: - Redactar **2 preguntas** sobre conceptos generales de transformaciones.

Responsabilidades en el deber: - Redactar **1 ejercicio conceptual** donde el estudiante deba identificar qué tipo de transformación aplicar en diferentes escenarios dados. —

PERSONA 2: TRANSFORMACIONES ESTRUCTURALES + CÓDIGO JAVA

Rol: Encargado de las transformaciones que cambian la forma del árbol.

Responsabilidades en la exposición: - Explicar **rotaciones** con diagramas (antes y después). - Explicar **conversión de árbol general a binario** (técnica hijo izquierdo – hermano derecho). - Mostrar la **lógica de programación** detrás de una rotación (reasignación de punteros). - Presentar **código en Java y C++** de una rotación simple (derecha o izquierda) aplicada a un árbol binario de búsqueda. - Hacer la transición a la Persona 3.

Responsabilidades en el Kahoot: - Redactar **2 preguntas** sobre rotaciones y conversión a binario, incluyendo interpretación de diagramas.

Responsabilidades en el deber: - Redactar **1 ejercicio práctico** donde se pida dibujar el resultado de aplicar una rotación específica sobre un árbol dado, o convertir un árbol general a su representación binaria.

PERSONA 3: TRANSFORMACIONES REDUCTIVAS + CÓDIGO C++

Rol: Encargado de las transformaciones que simplifican el árbol.

Responsabilidades en la exposición: - Explicar la **poda o pruning**: concepto, necesidad (evitar sobreajuste) y lógica de programación. - Explicar **fusión de nodos equivalentes**. - Mostrar cómo estas transformaciones son fundamentales en árboles de decisión. - Presentar **código en C++ y Java** de

una función de poda simple sobre un árbol de decisión. - Hacer la transición a la Persona 4.

Responsabilidades en el Kahoot: - Redactar **2 preguntas** sobre poda y fusión, con casos aplicados a árboles de decisión.

Responsabilidades en el deber: - Redactar **1 ejercicio** donde se muestre un árbol de decisión con sobreajuste y el estudiante deba identificar qué ramas se deben podar y justificar por qué.

PERSONA 4: ÁRBOL DE DECISIÓN + EJERCICIO PRÁCTICO + CÓDIGO JAVA Y C++

Rol: Encargado del subtema principal y la demostración interactiva.

Responsabilidades en la exposición: - Explicar **qué es un árbol de decisión** como estructura de datos (nodos-pregunta, ramas-respuesta, hojas-decisión). - Diferenciarlo del árbol binario de búsqueda. - Explicar la **lógica de construcción**: particionamiento recursivo. - **Realizar el ejercicio práctico con la clase**: Presentar un problema de decisión (ej: diagnóstico de fallas de un sistema) y construir el árbol en vivo con participación del curso. - Mostrar fragmentos de código en **Java** (construcción del árbol) y en **C++** (recorrido para clasificar un nuevo caso). - Hacer la transición a la Persona 5.

Responsabilidades en el Kahoot: - Redactar **2 preguntas** sobre la estructura del árbol de decisión y su construcción.

Responsabilidades en el deber: - Redactar **1 ejercicio de construcción** donde se den datos tabulares y el estudiante deba construir manualmente un árbol de decisión paso a paso.

PERSONA 5: CIERRE, KAHOOT Y DEBER

Rol: Encargado de la conclusión, la evaluación interactiva y la asignación del deber.

Responsabilidades en la exposición: - Hacer el **cierre de la clase**: resumir lo visto y conectar transformaciones con árbol de decisión. - Mostrar la **tabla comparativa** entre tipos de árboles. - Presentar las **conclusiones finales**. - **Dirigir el Kahoot** (o la herramienta elegida) con las 10 preguntas preparadas por todos. - Mostrar las respuestas correctas y dar **retroalimentación breve** en cada pregunta. - **Explicar el deber** y entregarlo formalmente. - Dar las gracias y abrir espacio para preguntas finales.

Responsabilidades en el Kahoot: - Recopilar las 8 preguntas de los compañeros y redactar **2 preguntas adicionales** (total: 10). - Sus 2 preguntas deben ser de integración (conectar transformaciones con árboles de decisión). - Crear el quizz en la plataforma elegida. - Conducir la dinámica en vivo.

Responsabilidades en el deber: - Compilar los ejercicios de todos en un solo documento. - Redactar **1 ejercicio integrador final** donde el estudiante deba aplicar una transformación (poda o conversión a reglas) sobre un árbol de decisión dado. - Dar formato al documento y entregarlo.

APE

Documento compilado por la Persona 5 con los ejercicios de todos:

Ejercicio 1 (Persona 1) – Identificación de transformaciones

Enunciado: Para cada escenario, indica qué tipo de transformación se debe aplicar y por qué: - a) Un árbol AVL después de insertar un nodo queda con factor de balance 2. - b) Un árbol de decisión tiene ramas con solo 1 ejemplo cada una. - c) Se necesita guardar un árbol en un archivo JSON.

Ejercicio 2 (Persona 2) – Rotaciones

Enunciado: Dado el siguiente árbol, dibuja el resultado de aplicar una rotación simple a la derecha sobre el nodo que contiene el valor 10.

Ejercicio 3 (Persona 3) – Poda

Enunciado: El siguiente árbol de decisión clasifica correos como spam o no spam. Identifica al menos dos ramas candidatas a ser podadas y justifica por qué.

Ejercicio 4 (Persona 4) – Construcción de árbol de decisión

Enunciado: Con los siguientes datos tabulares, construye manualmente un árbol de decisión paso a paso, indicando qué atributo elegiste en cada nodo y por qué.

Ejercicio 5 (Persona 5) – Integrador

Enunciado: Dado un árbol de decisión completo, aplica la transformación de conversión a reglas y escribe todas las reglas resultantes. Luego, identifica si alguna regla puede simplificarse.

RESUMEN DE RESPONSABILIDADES POR PERSONA

Persona	Rol	Exposición	Kahoot	APE
1	Coordinador y apertura	Inicio, contexto, clasificación de transformaciones, mapa conceptual	2 preguntas	1 ejercicio conceptual

Persona	Rol	Exposición	Kahoot	APE
2	Transformaciones, estructuras	Rotaciones, conversión a binario, lógica de punteros, código Java	2 preguntas	1 ejercicio de rotación
3	Transformaciones, reductivas	Pinches, fusión, sobreajuste, código C++	2 preguntas	1 ejercicio de poda
4	Árbol de decisión + ejercicio	Estructura, construcción, ejercicio interactivo, códigos Java y C++	2 preguntas	1 ejercicio de construcción
5	Cierre, Kahoot y deber	Conclusiones, tabla comparativa, dirige Kahoot, entrega deber	2 preguntas + plataforma	Ejercicio integrador + compilación